Harika Reddy

Embedded Software Engineer- Moog Inc.

Project Portfolio

Project 1

COMAC C919 Avionics :

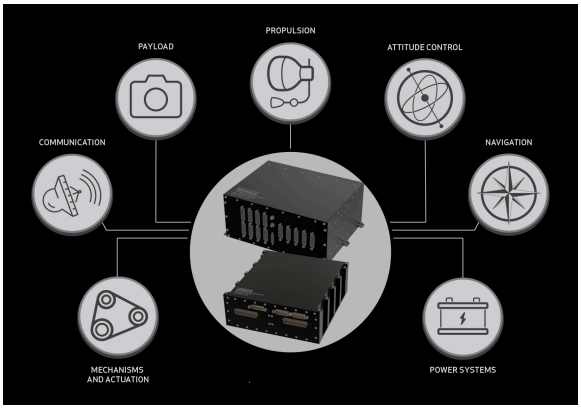
Developed embedded C++ firmware for the COMAC C919 commercial aircraft flight control system at Moog Inc. Worked on three Command and Control Modules — CCM1, CCM2, and CCM3 — safety-critical avionics computers physically installed inside the cockpit, positioned left, right, and bottom under the pilots' feet.

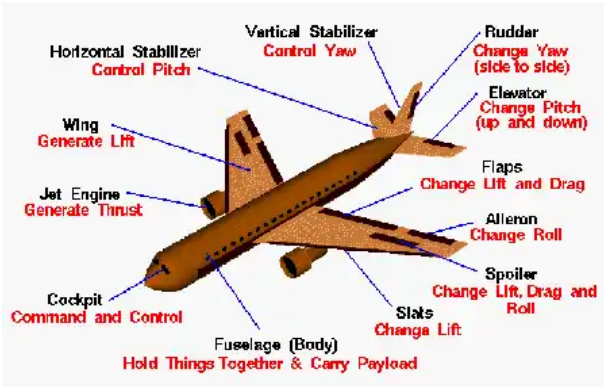
CCM1 acts as the primary controller computing all flight control commands, CCM2 runs in parallel as the secondary/redundant unit taking over instantly if CCM1 fails, and CCM3 handles sensor data acquisition — collecting live aircraft state from sensors and feeding it to CCM1 and CCM2 over CAN. Implemented communication protocol handling (CAN, UART, RS-422, SPI/I²C), flight control logic for primary surfaces (elevator, rudder, slats, flaps),

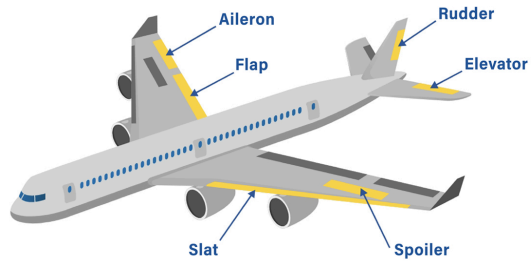
CCM1: primary

CCM2: Secondary / Redundant

CCM3: Sensor Data Acquisition







Goals :

1. **Safety** — if CCM1 fails, CCM2 takes over instantly so the aircraft never loses control.
2. **Accuracy** — CCM3 sensor data tells CCM1 exactly where each surface is so commands are precise.
3. **Cross-checking** — CCM1 and CCM2 compare outputs over CAN before any command reaches an actuator.
4. **Real-time** — FreeRTOS ensures pilot inputs are processed and executed in milliseconds.
5. **Sensor awareness** — CCM3 continuously feeds live aircraft state to CCM1 and CCM2 so they never command blindly.

Tools:

Language : C++

Compile Tool: Green hills Multi IDE

Validate Tool: MDTE (Moog Desktop Test Environment)

Hardware: Command and control modules.

Project 2

Vehicle Management System Computer (VMSC):

Developed embedded C firmware on STM32 ARM Cortex-M4 under FreeRTOS to drive the passenger display system on the VMSC project. Implemented task scheduling, queues, mutexes, and ISR handling to receive live aircraft sensor data over CAN and RS-422 and render flight information, entertainment controls, and safety messages on passenger screens.

Goals:

1. **Inform passengers** — show real-time flight data (altitude, speed, route map) collected from aircraft sensors
2. **Entertain passengers** — give passengers control over in-flight entertainment through the display
3. **Keep passengers safe** — instantly push safety and emergency messages to all screens when triggered
4. **Real-time updates** — ensure display always shows current data with no lag using FreeRTOS task scheduling
5. **Reliable delivery** — use queues and mutexes so sensor data never gets lost or corrupted between tasks before reaching the screen.

Tools:

Language : C

Tool: STM32CubeIDE

Hardware: STM32 ARM Cortex-M4

OS: FreeRTOS

Project 3

Multi-Motor Control Unit (MMCU) :

Developed embedded C firmware for the Multi Motor Control Unit — a defense-grade system responsible for controlling multiple motors simultaneously on a military platform. Worked on motor control logic including speed, position, and torque control algorithms, and performed rigorous verification and validation using IBM RTRT to meet defense safety standards.

Goals:

1. **Control multiple motors simultaneously** — firmware managed more than one motor at the same time with precise coordination
2. **Precision** — accurate speed, position, and torque control so motors respond exactly as commanded
3. **Real-time response** — motor commands executed immediately with no delay — critical in a defense environment
4. **Fault detection** — firmware detected motor faults and handled them safely without system failure
5. **Safety compliance** — all firmware verified and validated through IBM RTRT to meet defense software standards

Tools:

Language : C

Compile Tool: Multi IDE, RTRT tool

Hardware: NXP MPC5xxx

Project 4

Unmanned Aerial System (UAS):

The UAS project was in its early/initial development stage — no physical hardware existed yet. The flight control logic was being designed as **MATLAB Simulink models**. Your role was to write **Python test scripts** to validate these Simulink models through the **MITC automated test tool** — checking that the model logic was correct before any hardware or C code was developed.

Goals:

1. **Validate the model early** — catch logic errors in the Simulink model before any C code or hardware is built
2. **Automate testing** — Python scripts in MITC run tests automatically so every model change is checked instantly
3. **Save time and cost** — finding bugs at model stage is far cheaper than finding them after hardware is built
4. **Build test foundation** — Python test scripts written now become the basis for future hardware testing later

---

Project 5
ADAS (Advanced Driver Assistance Systems)

Performed model-based testing and SIL (Software-in-the-Loop) testing on MATLAB Simulink control models for Advanced Driver Assistance Systems at the initial stage of automotive product development. Validated lane assist and emergency braking control modules — feeding test inputs into Simulink models to verify correct logic, then validating the auto-generated C code using model based development tool to confirm the generated code behaved exactly as the model specified. All validation performed before any physical hardware was built, in compliance with ISO 26262 automotive safety standard.

Modules Validated:

- Lane Assist / Lane Keep
- Emergency Braking
- Adaptive Cruise Control

Goals :

1. **Prevent accidents** — validate lane assist and emergency braking logic catches dangerous situations correctly
2. **Early defect detection** — catch control logic errors at model stage before hardware is built
3. **Code correctness** — SIL testing confirms auto-generated C code behaves exactly as Simulink model intended
4. **ISO 26262 compliance** — all validation meets automotive functional safety standard

Tools:

Language : Matlab(Simulink), C

Tool: Matlab, Model-Based-Development.

Project 5
Chassis

Performed model-based testing and SIL (Software-in-the-Loop) testing on MATLAB Simulink control models for chassis dynamic control systems at the initial stage of automotive product development. Validated ABS and traction control modules — verifying that braking and wheel slip control logic was correct at model level, then confirming the auto-generated C code produced identical behaviour to the Simulink model through SIL testing using IBM RTRT. All validation completed at model and code level before hardware development began, ensuring early defect detection in compliance with ISO 26262.

Modules Validated:

- ABS (Anti-lock Braking System)
- Traction Control
- Electronic Stability Control (ESC)

Goals:

1. **Vehicle safety** — validate ABS and traction control logic correctly prevents wheel lock and wheel spin
2. **Stability** — ESC logic validated to keep vehicle stable during cornering and skidding
3. **Early defect detection** — catch braking and stability logic errors at model stage before hardware exists
4. **Code correctness** — SIL testing confirms auto-generated C code matches Simulink model behaviour exactly

Tools:

Language : Matlab(Simulink), C

Tool: Matlab, Model-Based-Development.